Modelo de conteo — ATUS ZMM

Primera escalera de modelos para predecir cuántos accidentes habrá en cada celda de la Zona Metropolitana de Monterrey y en qué mes: panel hexagonal, *baseline* histórico, GLM de Poisson, Binomial Negativa, diagnóstico de ceros y validación fuera de muestra en 2024. Es la parte frecuentista (pasos 1 a 4); el modelo jerárquico espacial (paso 5) va en un notebook aparte, con la especificación que **este** decide.

Fuente: `data/processed/atus_zmm_limpio_geo.parquet`, particionado con `geostats.espacial.celdas`, la misma rejilla de `analisis_espacial.qmd`.

```
uv run limpiar-atus --geo
```

Qué hay aquí

§	Tema
1	El panel hexágono × mes y los dos *baselines*
2	GLM de Poisson: documentar la sobredispersión
3	Binomial Negativa: corregir la varianza
4	Ceros: ¿hace falta un modelo aparte?
5	Validación en 2024
6	Dónde falla el modelo: el residual es municipal
7	La ventana del histórico
8	Qué tiene que ser el paso 5

> **Advertencia que atraviesa todo el documento.** Ningún modelo se compara en los datos con que se ajustó. Todas las métricas de desempeño son sobre 2024, que ningún modelo vio. Los AIC se reportan solo para comparar especificaciones **con la misma covariable de historia**; entre ventanas de historia distintas el AIC no es comparable, porque la covariable se construye a partir de la propia respuesta.

```
import warnings
warnings.filterwarnings("ignore")
from statsmodels.tools.sm_exceptions import HessianInversionWarning

import numpy as np
import pandas as pd
import geopandas as gpd
```

```
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, TwoSlopeNorm
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.discrete.count_model import ZeroInflatedNegativeBinomialP
import patsy
from libpysal.weights import lag_spatial
from esda.moran import Moran

from geostats import espacial, rutas

rng = np.random.default_rng(42)

g = gpd.read_parquet(rutas.ATUS_ZMM_LIMPIO_GEO).to_crs(espacial.UTM_ZMM)
N = len(g)
print(f"{N:,} registros | {g.fecha.min():%Y-%m-%d} a {g.fecha.max():%Y-%m-%d}")
```

```
379,294 registros | 2019-01-01 a 2024-12-31
```

Estilo de las gráficas

Misma identidad que los notebooks anteriores.

```
# --- Identidad GeoStats ------------------------------------------------------
AZUL     = "#005991"
ROJO     = "#8B2C1A"
OXIDO    = "#B15E2E"
GRAFITO  = "#2C2C2C"
GRIS     = "#F2F2F2"
BLANCO   = "#FFFFFF"
AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]

SECUENCIAL = LinearSegmentedColormap.from_list("geostats_seq", [BLANCO, AZUL])
DIVERGENTE = LinearSegmentedColormap.from_list("geostats_div", [AZUL, GRIS, ROJO])

TITULAR = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO   = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS  = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 16,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,
    "grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
```

```
    "xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.frameon": False,
    "legend.fontsize": 8.5, "lines.linewidth": 1.8,
})

def pulir(ax, ejes=("top", "right", "left")):
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax

def mapa(ax, titulo=None, escala_km=10):
    ax.set_aspect("equal"); ax.grid(False)
    ax.set_xticks([]); ax.set_yticks([])
    for lado in ax.spines.values():
        lado.set_visible(False)
    if titulo:
        ax.set_title(titulo)
    x0, x1 = ax.get_xlim(); y0, y1 = ax.get_ylim()
    bx = x0 + 0.04 * (x1 - x0); by = y0 + 0.04 * (y1 - y0)
    ax.plot([bx, bx + escala_km * 1000], [by, by], color=GRAFITO, lw=2, solid_capstyle="b
    ax.text(bx, by + 0.012 * (y1 - y0), f"{escala_km} km", fontsize=7.5,
            color=GRAFITO, fontfamily="monospace")
    return ax
```

---

1. El panel y los *baselines*

Las mismas celdas que el análisis espacial

`espacial.celdas` reproduce la partición del notebook anterior: hexágonos de 500 m, solo los que tienen al menos un accidente, sin islas. Se verifica que salen las mismas 2,270.

```
H, W = espacial.celdas(g, lado=500)
assert len(H) == 2270 and len(W.islands) == 0, "la partición cambió respecto a analisis_e

union = gpd.sjoin(g[["geometry", "ANIO", "MES", "NOM_MUN"]], H[["hex", "geometry"]],
                  predicate="within", how="inner")
ANIOS = list(range(2019, 2025))
indice = pd.MultiIndex.from_product([H.hex, ANIOS, range(1, 13)], names=["hex", "anio", "
p = (union.groupby(["hex", "ANIO", "MES"]).size()
     .reindex(indice, fill_value=0).rename("y").reset_index())

# municipio de cada celda: el modal entre sus accidentes
municipio = union.groupby("hex").NOM_MUN.agg(lambda s: s.mode().iloc[0]).rename("municipi
p = p.join(municipio, on="hex")

print(f"Panel: {len(p):,} filas = {len(H):,} celdas × {len(ANIOS) * 12} meses")
```

```
print(f"Suma de y: {p.y.sum():,} (los {N - p.y.sum():,} restantes son islas y un punto en
print(f"Filas en cero: {(p.y == 0).mean():.1%} | media {p.y.mean():.2f} | varianza {p.y.v
      f"| razón V/M = {p.y.var() / p.y.mean():.1f}")
```

```
Panel: 163,440 filas = 2,270 celdas × 72 meses
Suma de y: 379,160 (los 134 restantes son islas y un punto en arista)
Filas en cero: 56.9% | media 2.32 | varianza 30.03 | razón V/M = 12.9
```

Los ceros son observaciones. Una celda sin accidentes en un mes es un dato tan real como una con treinta; si se dejaran fuera, el modelo aprendería solo de los meses con eventos y sobreestimaría todo. Por eso el panel se construye completo y se rellena con cero. El 57 % de las filas lo son.

La razón varianza/media de 12.9 es la sobredispersión **marginal**: antes de cualquier covariable. Buena parte es heterogeneidad entre celdas (una celda con media 50 y otra con media 0.1 en el mismo panel inflan la varianza); lo que quede *después* de condicionar en la celda es lo que decide entre Poisson y Binomial Negativa.

Partición temporal y covariables

```
p["periodo"] = np.select(
    [(p.anio == 2019) | ((p.anio == 2020) & (p.mes <= 2)), p.anio <= 2021],
    ["prepandemia", "pandemia"], default="georef2022")

tr = p[p.anio <= 2023].copy()
te = p[p.anio == 2024].copy()

# Historia de cada celda, calculada SOLO con el periodo de entrenamiento.
h_completa = tr.groupby("hex").y.mean().rename("h")
h_reciente = tr[tr.anio >= 2022].groupby("hex").y.mean().rename("h_rec")
for d in (tr, te):
    d["h"] = d.hex.map(h_completa)
    d["h_rec"] = d.hex.map(h_reciente)
    d["logh"] = np.log(d.h + 0.5 / 60)
    d["logh_rec"] = np.log(d.h_rec + 0.5 / 24)

print(f"Entrenamiento 2019–2023: {len(tr):,} filas | Prueba 2024: {len(te):,} filas")
print(tr.periodo.value_counts().rename("filas por periodo").to_string())
```

```
Entrenamiento 2019–2023: 136,200 filas | Prueba 2024: 27,240 filas
periodo
georef2022     54480
pandemia       49940
prepandemia    31780
```

Tres covariables, todas estructurales:

- `mes` como factor: la estacionalidad que el STL del análisis multivariado ya mostró.
- `periodo` en tres niveles: prepandemia, pandemia (marzo 2020 a diciembre

2021. y 2022 en adelante. El tercer corte es el cambio de georreferenciación detectado en el análisis espacial (`OID` aparece en 2022). Sin él, el modelo leería el salto como tendencia.

- `log h`: el promedio mensual histórico de la celda, calculado solo con entrenamiento. Es la versión de bolsillo de un efecto por celda: en vez de 2,270 parámetros, uno solo que dice cuánto pesa la historia propia. Su coeficiente es interpretable: **1 significa "la historia entra como offset"**; menos de 1 significaría que el modelo encoge las celdas hacia la media.

Los dos *baselines*

Antes de cualquier modelo, dos predictores sin parámetros: la media mensual de la celda en 2019-2023 (`h`), y la de 2022-2023 (`h_rec`). El primero es el PAI 8.4 del análisis espacial; el segundo, la apuesta de que lo reciente pesa más.

```
y24_hex = te.groupby("hex").y.sum()

def dist_de(mu, theta=None):
    """Poisson, o Binomial Negativa parametrizada por media y theta."""
    return stats.poisson(mu) if theta is None else stats.nbinom(n=theta, p=theta / (theta

def pai(pred_hex, y_hex, f):
    k = max(1, int(len(pred_hex) * f))
    top = pred_hex.nlargest(k).index
    return (y_hex.loc[top].sum() / y_hex.sum()) / f

def evaluar(nombre, mu, theta=None, K=400):
    """Métricas fuera de muestra sobre 2024. `mu` alineado con `te`."""
    y = te.y.values
    dist = dist_de(mu, theta)
    devianza = 2 * np.sum(np.where(y > 0, y * np.log(np.where(y > 0, y, 1) / mu), 0) - (y
    pred_hex = pd.Series(mu, index=te.index).groupby(te.hex).sum()
    lo, hi = dist.ppf(0.05), dist.ppf(0.95)
    ks = np.arange(K)
    F = dist_de(mu[:, None], theta).cdf(ks[None, :])
    escalon = (y[:, None] <= ks[None, :]).astype(float)
    return {"modelo": nombre,
            "devianza": devianza,
            "MAE": np.mean(np.abs(y - mu)),
            "CRPS": np.mean(np.sum((F - escalon) ** 2, axis=1)),
            "cobertura 90 %": np.mean((y >= lo) & (y <= hi)),
            "PAI 5 %": pai(pred_hex, y24_hex, 0.05),
            "PAI 10 %": pai(pred_hex, y24_hex, 0.10)}

resultados = []
resultados.append(evaluar("M0  histórico 2019–23", te.h.values + 0.5 / 60))
resultados.append(evaluar("M0r histórico 2022–23", te.h_rec.values + 0.5 / 24))
pd.DataFrame(resultados).set_index("modelo").round(3)
```

	devianza	MAE	CRPS	cobertura 90 %	PAI 5 %	PAI 10 %
modelo						
M0 histórico 2019-23	40949.315	1.232	0.899	0.898	8.443	5.846
M0r histórico 2022-23	35842.617	1.184	0.835	0.920	8.424	5.961

Las métricas, en una frase cada una: la **devianza de Poisson** y el **MAE** miden qué tan cerca está la media predicha; el **CRPS** mide qué tan buena es la *distribución* predicha entera (penaliza tanto errar la media como errar la incertidumbre); la **cobertura** dice si el intervalo predictivo del 90 % contiene al 90 % de lo observado; y el **PAI** es la métrica operativa: cuántas veces mejor que el azar es elegir el top 5 % o 10 % de celdas con este modelo.

Ya se ve algo: la historia reciente predice mejor en todo salvo en el PAI 5 %, donde empata. Se vuelve a esto en la §7.

---

2. GLM de Poisson

Método. Modelo lineal generalizado con respuesta Poisson y enlace logarítmico, estimado por máxima verosimilitud:

$$Y_{it} \sim \text{Poisson}(\mu_{it}), \qquad \log \mu_{it} = \beta_0 + \gamma_{\text{mes}(t)} + \delta_{\text{periodo}(t)} + \beta_h \log h_i$$

Supuestos. (1) Equidispersión, $\text{Var}(Y) = \mu$; (2) independencia condicional entre observaciones; (3) efectos multiplicativos (log-lineal); (4) los eventos no se causan entre sí.

Se ajusta sabiendo que el primero va a fallar. Los coeficientes de Poisson son consistentes aunque haya sobredispersión —lo que se rompe son sus errores estándar—, así que el modelo sirve de referencia y de diagnóstico.

```
FORMULA = "y ~ C(mes) + C(periodo) + logh"
m_pois = smf.glm(FORMULA, data=tr, family=sm.families.Poisson()).fit()

print(f"Coeficiente de log h: {m_pois.params['logh']:.3f}   (1 = la historia es un offset
print(f"Periodo — pandemia: ×{np.exp(m_pois.params['C(periodo)[T.pandemia]']):.3f}   "
      f"prepandemia: ×{np.exp(m_pois.params['C(periodo)[T.prepandemia]']):.3f}   (base: 2
print(f"χ² de Pearson / gl: {m_pois.pearson_chi2 / m_pois.df_resid:.3f}   (1 bajo equidis
```

```
Coeficiente de log h: 1.007   (1 = la historia es un offset)
Periodo — pandemia: ×0.785   prepandemia: ×0.960   (base: 2022+)
χ² de Pearson / gl: 1.368   (1 bajo equidispersión)
```

Prueba de sobredispersión (Cameron–Trivedi)

La prueba ajusta por mínimos cuadrados la regresión auxiliar $\frac{(y-\hat\mu)^2-y}{\hat\mu} = \alpha\,\hat\mu + \varepsilon$. Bajo Poisson $\alpha = 0$; un $\alpha > 0$ significativo es la firma de la varianza cuadrática de la Binomial Negativa.

```
mu_tr = m_pois.fittedvalues.values
y_tr = tr.y.values
auxiliar = ((y_tr - mu_tr) ** 2 - y_tr) / mu_tr
ct = sm.OLS(auxiliar, mu_tr).fit()
print(f"α̂ = {ct.params[0]:.4f}   t = {ct.tvalues[0]:.1f}   p = {ct.pvalues[0]:.1e}")
```

```
α̂ = 0.0957   t = 59.7   p = 0.0e+00
```

De 12.9 marginal a 1.37 condicional: `log h` **absorbe casi toda la sobredispersión**, porque la mayor parte era heterogeneidad entre celdas. Pero lo que queda es inequívoco: $t = 60$. La varianza de cada celda-mes sigue siendo un 37 % mayor que su media, y Poisson lo ignora.

---

3. Binomial Negativa

Método. Mezcla Poisson–Gamma (NB2). La tasa de cada celda-mes no es μ_{it} exactamente sino una variable Gamma alrededor de μ_{it}; los factores que no observamos —obra, semáforo descompuesto, lluvia— viven ahí:

$$Y_{it} \mid \lambda \sim \text{Poisson}(\lambda), \quad \lambda \sim \text{Gamma}\left(\theta, \tfrac{\mu_{it}}{\theta}\right) \implies \text{Var}(Y_{it}) = \mu_{it} + \frac{\mu_{it}^2}{\theta}$$

Supuestos. (1) La heterogeneidad no observada es Gamma; (2) θ es constante (NB2, varianza cuadrática en μ; la alternativa NB1 la hace lineal); (3) independencia condicional; (4) log-linealidad.

```
m_nb2 = smf.negativebinomial(FORMULA, data=tr).fit(disp=0, maxiter=200)
m_nb1 = smf.negativebinomial(FORMULA, data=tr, loglike_method="nb1").fit(disp=0, maxiter=
theta = 1 / m_nb2.params["alpha"]

lr = 2 * (m_nb2.llf - m_pois.llf)
print(f"NB2: θ = {theta:.2f}   coef. log h = {m_nb2.params['logh']:.3f}")
print(f"AIC — Poisson {m_pois.aic:,.0f} | NB1 {m_nb1.aic:,.0f} | NB2 {m_nb2.aic:,.0f}")
print(f"Razón de verosimilitudes Poisson vs NB2: {lr:,.0f}   p(mezcla χ²) = {0.5 * stats.
```

```
NB2: θ = 5.54   coef. log h = 1.011
AIC — Poisson 327,634 | NB1 315,000 | NB2 314,354
Razón de verosimilitudes Poisson vs NB2: 13,282   p(mezcla χ²) = 0.0e+00
```

Tres lecturas:

- **La prueba de razón de verosimilitudes no es una χ^2_1 ordinaria.** La hipótesis nula $1/\theta = 0$ está en la frontera del espacio de parámetros (la dispersión no puede ser negativa), y la distribución nula correcta es una mezcla 50:50 de una masa en cero y una χ^2_1. El *p* correcto es la mitad del de tabla. Aquí da igual: 13,282 contra un umbral de ~2.7.
- **NB2 sobre NB1** por 600 puntos de AIC: la varianza crece con el cuadrado de la media, no linealmente. Es lo usual en siniestralidad vial.

- **El coeficiente de `log h` sigue siendo 1.01.** El modelo no encoge: cada celda entra con su historia completa. Eso tiene una consecuencia que la §5 hace visible.

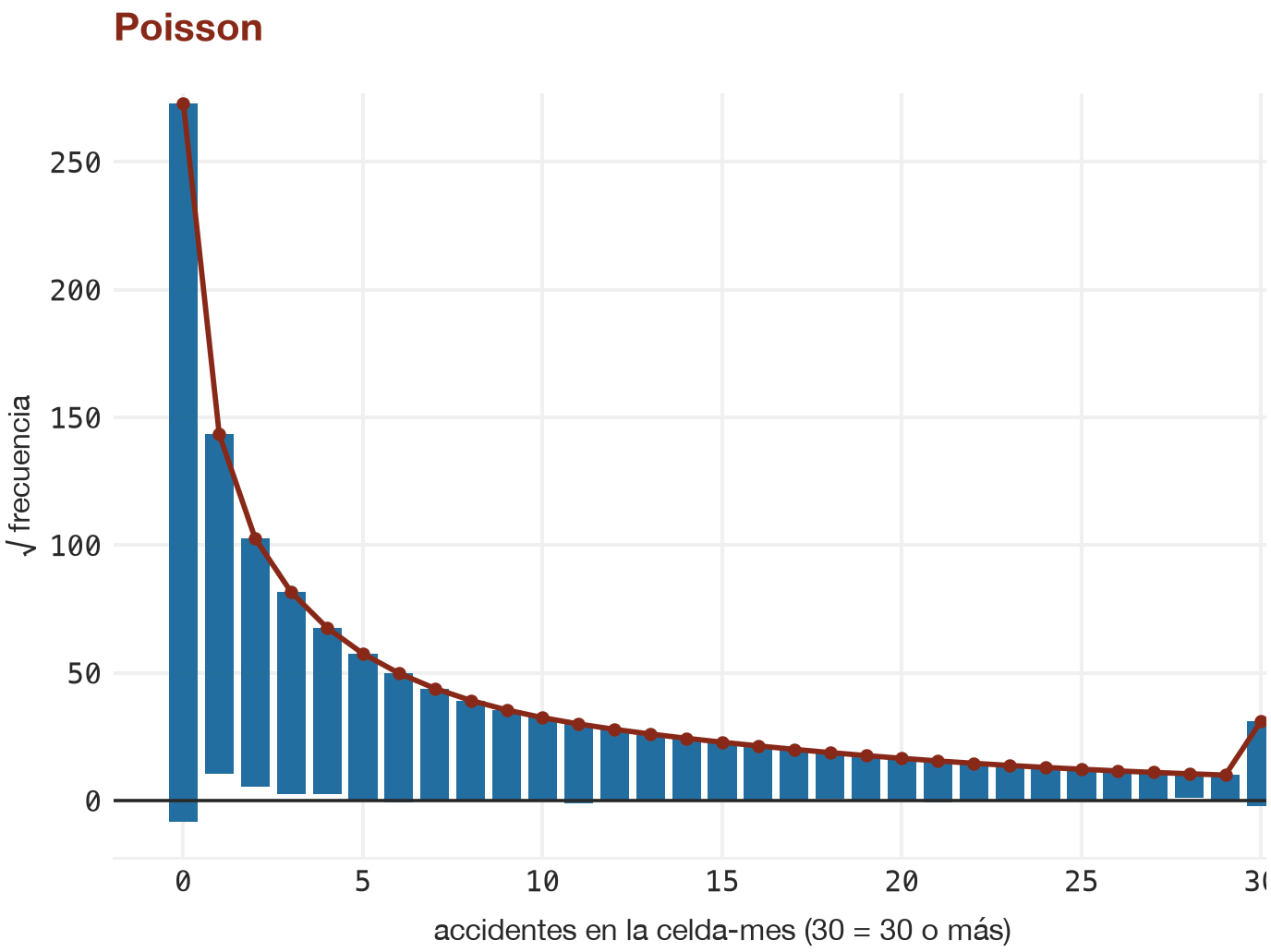
Rootograma colgante

El diagnóstico visual para conteos (Kleiber & Zeileis 2016): para cada valor k se dibuja la raíz de la frecuencia esperada como curva, y de ella *cuelga* una barra de altura raíz de la observada. Si el modelo ajusta, todas las barras tocan el cero. Barras que flotan son valores sobrepredichos; barras que atraviesan el eje son subpredichos.

```
K = 30
obs = np.bincount(np.minimum(y_tr, K), minlength=K + 1)
mu_nb_tr = m_nb2.predict(tr).values

def esperados(dist):
    e = np.array([dist.pmf(k).sum() for k in range(K)])
    return np.append(e, len(y_tr) - e.sum())

fig, ejes = plt.subplots(1, 2, figsize=(11, 4.2), sharey=True)
for ax, (nombre, dist) in zip(ejes, [("Poisson", dist_de(mu_tr)), (f"Binomial Negativa (θ
    esp = esperados(dist)
    ks = np.arange(K + 1)
    ax.bar(ks, -np.sqrt(obs), bottom=np.sqrt(esp), width=0.8, color=AZUL, alpha=0.85, lin
    ax.plot(ks, np.sqrt(esp), color=ROJO, lw=1.6, marker="o", ms=3)
    ax.axhline(0, color=GRAFITO, lw=1)
    ax.set_title(nombre)
    ax.set_xlabel("accidentes en la celda-mes (30 = 30 o más)")
    pulir(ax)
ejes[0].set_ylabel("√ frecuencia")
plt.tight_layout(); plt.show()
```



Rootogramas colgantes en entrenamiento. Raíz cuadrada para que la cola sea visible junto a los ceros.

```
esp_p, esp_n = esperados(dist_de(mu_tr)), esperados(dist_de(mu_nb_tr, theta))
tabla_k = pd.DataFrame({"observados": obs, "Poisson": esp_p, "NB2": esp_n}, index=pd.Inde
tabla_k.loc[[0, 1, 2, 3, 5, 10, 20, 30]].round(0).astype(int)
```

	observados	Poisson	NB2
k			
0	79148	74579	76585
1	17671	20552	20231
2	9405	10533	10130
3	6248	6665	6327
5	3295	3312	3139
10	1023	1056	1008
20	261	275	261
30	1118	975	1202

Poisson subpredice los ceros en 4,600 y sobrepredice los unos en 2,900: el patrón clásico de un modelo con la varianza demasiado angosta. NB2 lo corrige en buena parte, pero **no del todo**: sigue faltando el 3.2 % de los ceros. Ese residuo es lo que decide la sección siguiente.

---

4. Ceros: ¿hace falta un modelo aparte?

Los modelos de inflación de ceros (ZIP, ZINB) suponen **dos procesos**: una moneda que decide si la observación *puede* tener eventos (cero estructural) y, si puede, un conteo (cero muestral). Se justifican solo si los ceros estructurales existen.

La partición ya retiró las celdas sin red vial, que eran el cero estructural obvio. Pero el rootograma dice que quedan más ceros de los que NB2 explica. Se ajusta un ZINB con probabilidad de inflación constante para medir cuántos.

```
X = patsy.dmatrix("C(mes) + C(periodo) + logh", tr, return_type="dataframe")
m_zinb = ZeroInflatedNegativeBinomialP(tr.y.values, X, exog_infl=np.ones((len(tr), 1)), p
                                       ).fit(disp=0, maxiter=500, method="bfgs")
pi_inf = 1 / (1 + np.exp(-m_zinb.params["inflate_const"]))
print(f"Probabilidad de cero estructural π̂ = {pi_inf:.3f}   ({pi_inf * len(tr):,.0f} fila
print(f"AIC — NB2 {m_nb2.aic:,.0f} | ZINB {m_zinb.aic:,.0f}   (Δ = {m_nb2.aic - m_zinb.ai
print(f"Convergió: {m_zinb.mle_retvals['converged']}")
```

```
Probabilidad de cero estructural π̂ = 0.026   (3,570 filas)
AIC — NB2 314,354 | ZINB 313,173   (Δ = 1,182)
Convergió: True
```

Un 2.6 % de ceros estructurales, 3,600 filas. Es real —el AIC mejora 1,200— y es pequeño. La pregunta útil no es si ponerle un modelo, sino **de dónde salen**. Si fueran celdas dispersas, ZINB sería la respuesta. Si estuvieran concentradas en un lugar y un tiempo, serían otra cosa.

```
tr["esperado_cero_nb"] = dist_de(mu_nb_tr, theta).pmf(0)
ceros = (tr.assign(cero=(tr.y == 0))
         .groupby(["municipio", "periodo"])
         .agg(filas=("y", "size"), ceros_obs=("cero", "sum"), ceros_esp=("esperado_cero_n
ceros["exceso"] = ceros.ceros_obs - ceros.ceros_esp
ceros["exceso_rel"] = ceros.exceso / ceros.filas
top_exceso = ceros.sort_values("exceso", ascending=False).head(8)
top_exceso.round(2)
```

		filas	ceros_obs	ceros_esp	exceso	exceso_rel
municipio	**periodo**					
Santa Catarina	**pandemia**	2112	2078	1348.98	729.02	0.35
	prepandemia	1344	1260	827.15	432.85	0.32
General Escobedo	**prepandemia**	2184	1402	1072.01	329.99	0.15

		filas	ceros_obs	ceros_esp	exceso	exceso_rel
municipio	**periodo**					
Guadalupe	**georef2022**	3888	1307	1058.44	248.56	0.06
Monterrey	**georef2022**	7776	2117	1888.76	228.24	0.03
Apodaca	**georef2022**	5568	2911	2696.91	214.09	0.04
General Escobedo	**pandemia**	3432	1967	1772.19	194.81	0.06
Pesquería	**pandemia**	2442	2120	1942.44	177.56	0.07

```
total_exceso = ceros.exceso.clip(lower=0).sum()
print(f"Exceso total de ceros: {ceros.exceso.sum():,.0f}")
print(f"Los 8 pares municipio×periodo de arriba explican el "
      f"{top_exceso.exceso.sum() / total_exceso:.0%} del exceso positivo")
```

```
Exceso total de ceros: 2,563
Los 8 pares municipio×periodo de arriba explican el 65% del exceso positivo
```

Los ceros estructurales no están dispersos: están en municipios y periodos concretos. Ocho pares municipio × periodo explican dos tercios del exceso, y Santa Catarina en prepandemia y pandemia encabeza la lista: el municipio registró 128, 3 y 77 accidentes en 2019, 2020 y 2021, y 1,795 en 2022. Esas celdas no tenían "riesgo cero": tenían un municipio que no reportaba. Un ZINB con inflación constante repartiría esa probabilidad entre todas las celdas por igual, que es exactamente lo contrario de lo que pasa.

Decisión: no se adopta ZINB. Los ceros en exceso son un fenómeno de *régimen de reporte por municipio*, y ese es un término del modelo jerárquico, no una segunda ecuación de ceros.

---

5. Validación en 2024

Todos los modelos ajustados con 2019-2023 predicen los 12 meses de 2024 sin haberlos visto. Se añade un cuarto modelo, NB2 con el rezago espacial de la historia (el promedio de `h` en las seis vecinas), para adelantar una pregunta del paso 5: ¿la vecindad aporta algo que la celda sola no tenga?

```
lag_h = pd.Series(lag_spatial(W, H.hex.map(h_completa).fillna(0).values), index=H.hex)
for d in (tr, te):
    d["loglagh"] = np.log(d.hex.map(lag_h) + 0.5 / 60)
m_nb2_lag = smf.negativebinomial(FORMULA + " + loglagh", data=tr).fit(disp=0, maxiter=200
theta_lag = 1 / m_nb2_lag.params["alpha"]
print(f"NB2 + rezago: coef. log h = {m_nb2_lag.params['logh']:.3f}, "
      f"coef. rezago = {m_nb2_lag.params['loglagh']:.3f} (t = {m_nb2_lag.tvalues['loglagh
```

```
NB2 + rezago: coef. log h = 1.013, coef. rezago = −0.002 (t = −0.5)
```

```
mu_pois_te = m_pois.predict(te).values
mu_nb2_te = m_nb2.predict(te).values
mu_lag_te = m_nb2_lag.predict(te).values

resultados.append(evaluar("M1  Poisson", mu_pois_te))
resultados.append(evaluar("M2  NB2", mu_nb2_te, theta))
resultados.append(evaluar("M3  NB2 + rezago espacial", mu_lag_te, theta_lag))
tabla = pd.DataFrame(resultados).set_index("modelo")
tabla.style.format({"devianza": "{:,.0f}", "MAE": "{:.3f}", "CRPS": "{:.3f}",
                    "cobertura 90 %": "{:.1%}", "PAI 5 %": "{:.2f}", "PAI 10 %": "{:.2f}"
```

	devianza	MAE	CRPS	cobertura 90 %	PAI 5 %	PAI 10 %
modelo						
M0 histórico 2019-23	40,949	1.232	0.899	89.8%	8.44	5.85
M0r histórico 2022-23	35,843	1.184	0.835	92.0%	8.42	5.96
M1 Poisson	39,917	1.233	0.885	90.3%	8.44	5.85
M2 NB2	39,914	1.240	0.876	93.9%	8.44	5.85
M3 NB2 + rezago espacial	39,913	1.240	0.876	93.9%	8.44	5.85

```
rho = stats.spearmanr(pd.Series(mu_nb2_te, index=te.index).groupby(te.hex).sum(), te.grou
print(f"Correlación de rangos entre el ranking de celdas de NB2 y el del histórico: {rho:
```

```
Correlación de rangos entre el ranking de celdas de NB2 y el del histórico: 1.0000
```

Lo que la tabla dice, y lo que no:

- **El PAI es idéntico en M0, M1, M2 y M3.** No es coincidencia: con $\beta_h \approx 1$ y covariables que solo varían en el tiempo, la predicción de cada celda es $h_i \times$ (un factor común a todas). El ranking es el del histórico, exactamente (Spearman = 1.0). **Un GLM con covariables temporales no puede mover el *dónde*; solo el *cuánto* y el *con qué certeza*.**
- Lo que sí mejora es la distribución: CRPS 0.899 → 0.876 de M0 a NB2, y la cobertura del 90 % sube de 89 % a 94 %. La NB2 sobrecubre un poco; es el precio de la discreción (con tantos ceros, el intervalo $[0, q_{95}]$ contiene más del 90 % siempre).
- **El rezago espacial no aporta nada**: coeficiente −0.002, $t = -0.5$. Dado el I de Moran de 0.65 sobre los conteos, esto sorprende — hasta que se mira dónde está el residual.

---

6. Dónde falla el modelo: el residual es municipal

Residuales cuantil aleatorizados

Los residuales de Pearson o de devianza no sirven para conteos con muchos ceros: son discretos y sesgados por construcción. Los **residuales cuantil aleatorizados** (Dunn & Smyth 1996) resuelven eso:

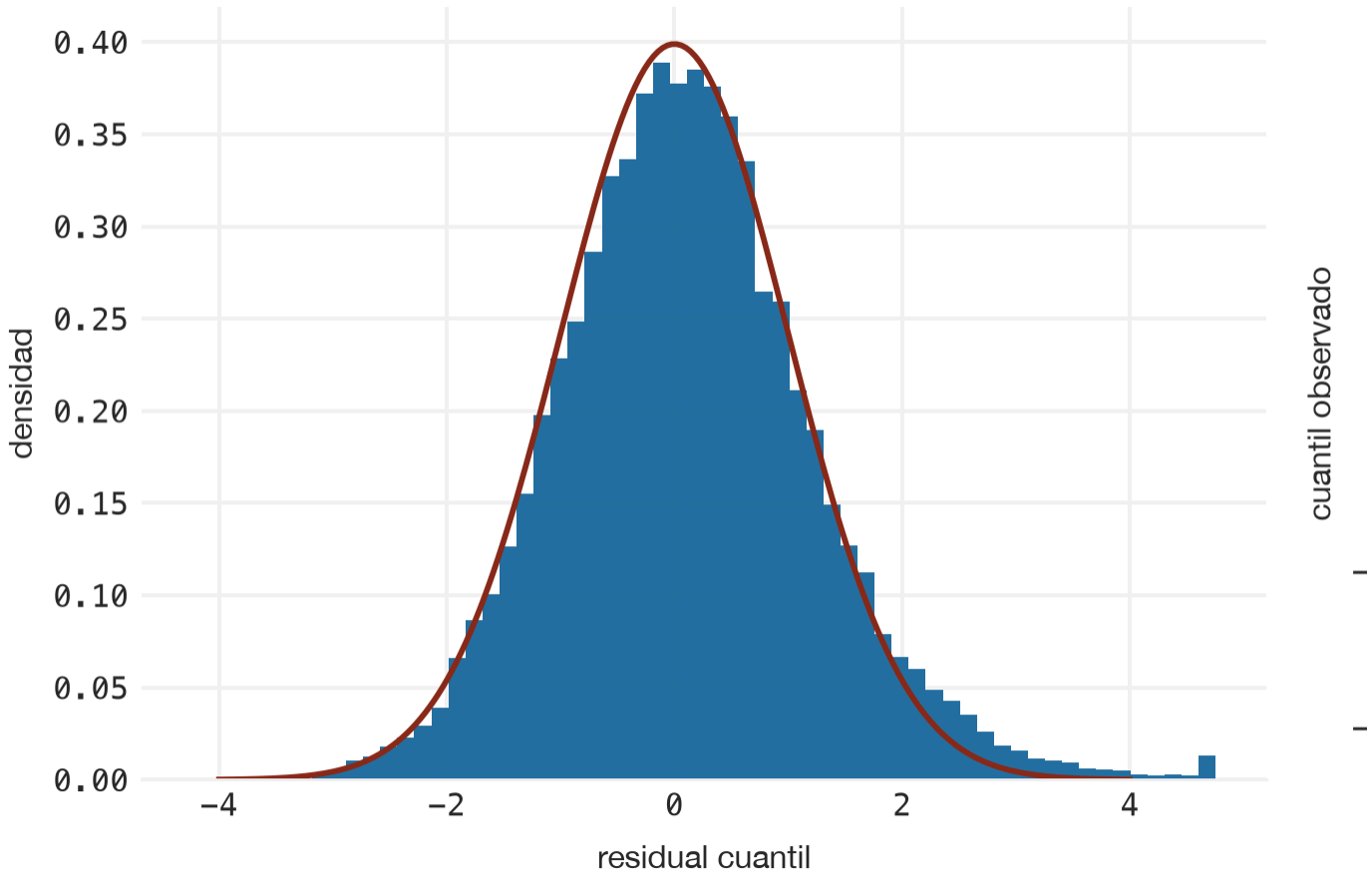
para cada observación se toma un punto uniforme entre $F(y-1)$ y $F(y)$ y se pasa por la normal inversa. Si el modelo es correcto, son exactamente $\mathcal{N}(0,1)$, sin importar la distribución de Y.

```
def rqr(y, dist):
    a, b = dist.cdf(y - 1), dist.cdf(y)
    u = np.clip(a + rng.uniform(size=len(y)) * (b - a), 1e-6, 1 - 1e-6)
    return stats.norm.ppf(u)

r_nb2 = rqr(te.y.values, dist_de(mu_nb2_te, theta))
print(f"RQR de NB2 en 2024: media {r_nb2.mean():.3f}, desviación {r_nb2.std():.3f}   (0 y
```

```
RQR de NB2 en 2024: media 0.110, desviación 1.114   (0 y 1 si el modelo es correcto)
```

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 4))
a1.hist(r_nb2, bins=60, density=True, color=AZUL, alpha=0.85, linewidth=0)
xs = np.linspace(-4, 4, 200)
a1.plot(xs, stats.norm.pdf(xs), color=ROJO, lw=1.6)
a1.set_xlabel("residual cuantil"); a1.set_ylabel("densidad"); a1.set_title("Distribución"
pulir(a1)
(osm, osr), _ = stats.probplot(r_nb2, dist="norm")
a2.scatter(osm, osr, s=3, color=AZUL, alpha=0.4, linewidths=0)
a2.plot([-4, 4], [-4, 4], color=ROJO, lw=1.2)
a2.set_xlabel("cuantil teórico N(0,1)"); a2.set_ylabel("cuantil observado"); a2.set_title
pulir(a2)
plt.tight_layout(); plt.show()
```



Residuales cuantil aleatorizados de NB2 sobre 2024. Deberían ser N(0,1); la desviación de 1.12 es dispersión que el modelo no captura.

El residual en el mapa

Se promedia el residual de cada celda sobre los doce meses de 2024 y se calcula el I de Moran. Si el modelo capturó toda la estructura espacial, el residual no debería tenerla.

```
r_hex = pd.Series(r_nb2, index=te.index).groupby(te.hex).mean().reindex(H.hex)
H["residual"] = r_hex.values
H["municipio"] = H.hex.map(municipio)
H["residual_intra"] = H.residual - H.groupby("municipio").residual.transform("mean")

mor_bruto = Moran(H.residual.values, W, permutations=999)
mor_intra = Moran(H.residual_intra.values, W, permutations=999)
ss_total = ((H.residual - H.residual.mean()) ** 2).sum()
ss_intra = (H.residual_intra ** 2).sum()

print(f"I de Moran del residual por celda:      {mor_bruto.I:.3f}   (z = {mor_bruto.z_sim
print(f"…quitando la media de cada municipio:   {mor_intra.I:.3f}   (z = {mor_intra.z_sim
print(f"Fracción de la varianza del residual que es entre municipios: {1 - ss_intra / ss_
```

```
I de Moran del residual por celda:      0.303   (z = 20.2)
…quitando la media de cada municipio:   0.056   (z = 3.8)
Fracción de la varianza del residual que es entre municipios: 25.7%
```

```
fig, ax = plt.subplots(figsize=(8.5, 8))
norma = TwoSlopeNorm(vmin=-1.2, vcenter=0, vmax=1.2)
H.plot(column="residual", cmap=DIVERGENTE, norm=norma, ax=ax, linewidth=0, legend=True,
       legend_kwds={"shrink": 0.5, "label": "residual cuantil medio", "pad": 0.01})
mapa(ax, "El error del modelo tiene forma de municipio")
plt.tight_layout(); plt.show()
```

El error del modelo tiene forma de municipio



Residual medio por celda en 2024, NB2. Azul: el modelo predijo de más; rojo: de menos. Los bloques siguen límites municipales.

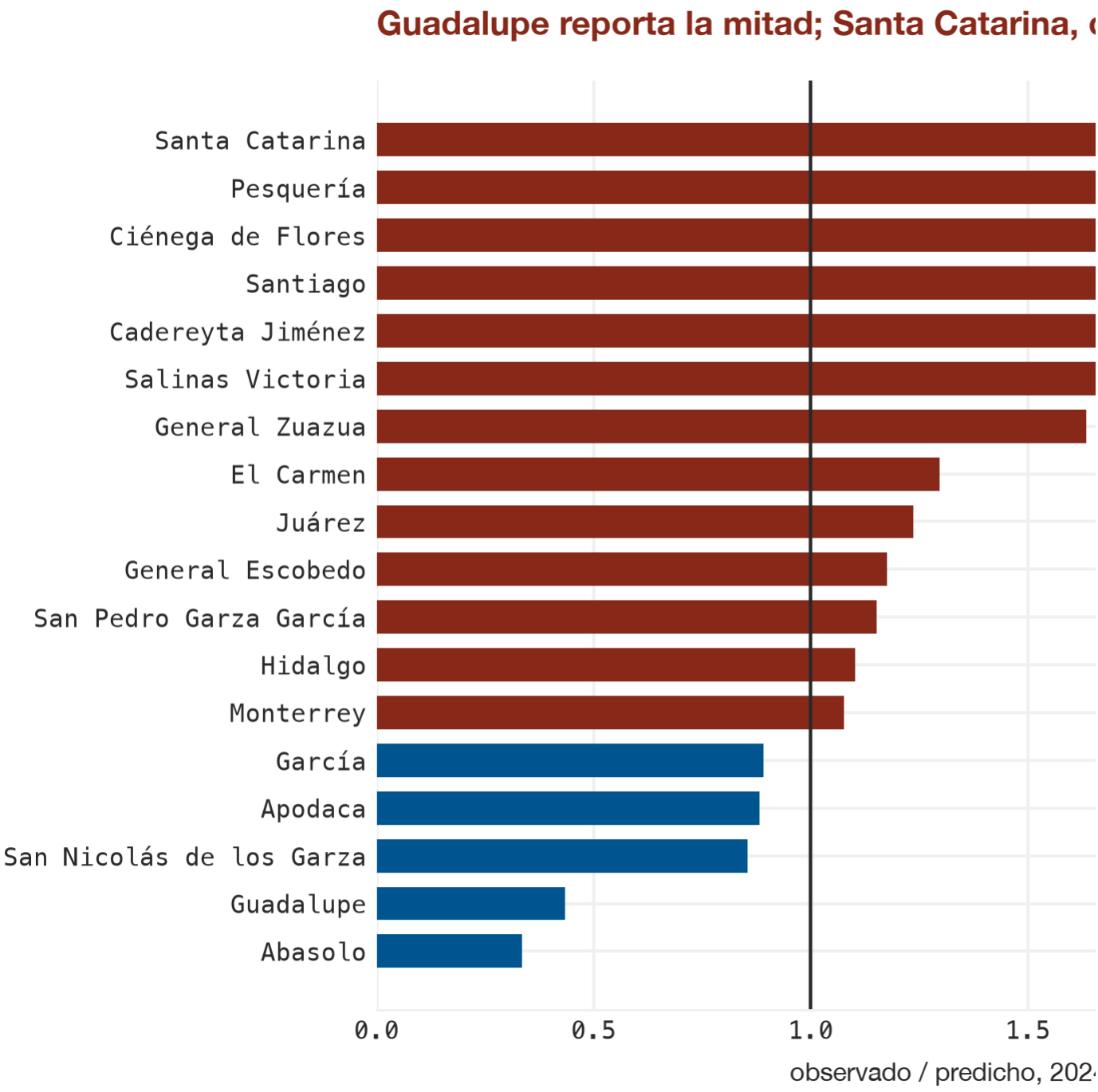
El I de Moran del residual cae de 0.30 a 0.06 al quitar la media municipal. Casi toda la autocorrelación que queda en el error no es un proceso espacial fino que un efecto BYM2 a 500 m pueda capturar: son 18 bloques con un nivel distinto cada uno. Por eso el rezago espacial no ayudó: las vecinas de una celda de Guadalupe también están en Guadalupe y se equivocan igual.

Observado contra predicho por municipio

```
por_mun = (pd.DataFrame({"y": te.y.values, "mu": mu_nb2_te, "municipio": te.municipio.val
           .groupby("municipio").agg(observado=("y", "sum"), predicho=("mu", "sum")))
por_mun["razón"] = por_mun.observado / por_mun.predicho
por_mun = por_mun.sort_values("razón")
por_mun.style.format({"observado": "{:,.0f}", "predicho": "{:,.0f}", "razón": "{:.2f}"})
```

	observado	predicho	razón
municipio			
Abasolo	3	9	0.34
Guadalupe	2,770	6,375	0.43
San Nicolás de los Garza	9,226	10,794	0.85
Apodaca	3,618	4,104	0.88
García	2,394	2,687	0.89
Monterrey	33,116	30,748	1.08
Hidalgo	92	83	1.10
San Pedro Garza García	6,878	5,976	1.15
General Escobedo	3,422	2,912	1.17
Juárez	1,743	1,410	1.24
El Carmen	184	142	1.30
General Zuazua	695	425	1.63
Salinas Victoria	819	465	1.76
Cadereyta Jiménez	630	342	1.84
Santiago	1,153	582	1.98
Ciénega de Flores	713	355	2.01
Pesquería	1,092	543	2.01
Santa Catarina	2,747	1,095	2.51

```
fig, ax = plt.subplots(figsize=(7.5, 5.5))
colores = [ROJO if r > 1 else AZUL for r in por_mun["razón"]]
ax.barh(por_mun.index, por_mun["razón"], color=colores, height=0.7)
ax.axvline(1, color=GRAFITO, lw=1.2)
ax.set_xlabel("observado / predicho, 2024")
ax.set_title("Guadalupe reporta la mitad; Santa Catarina, dos veces y media")
pulir(ax)
plt.tight_layout(); plt.show()
```



Cociente observado/predicho en 2024 por municipio. Un modelo correcto daría 1 en todos.

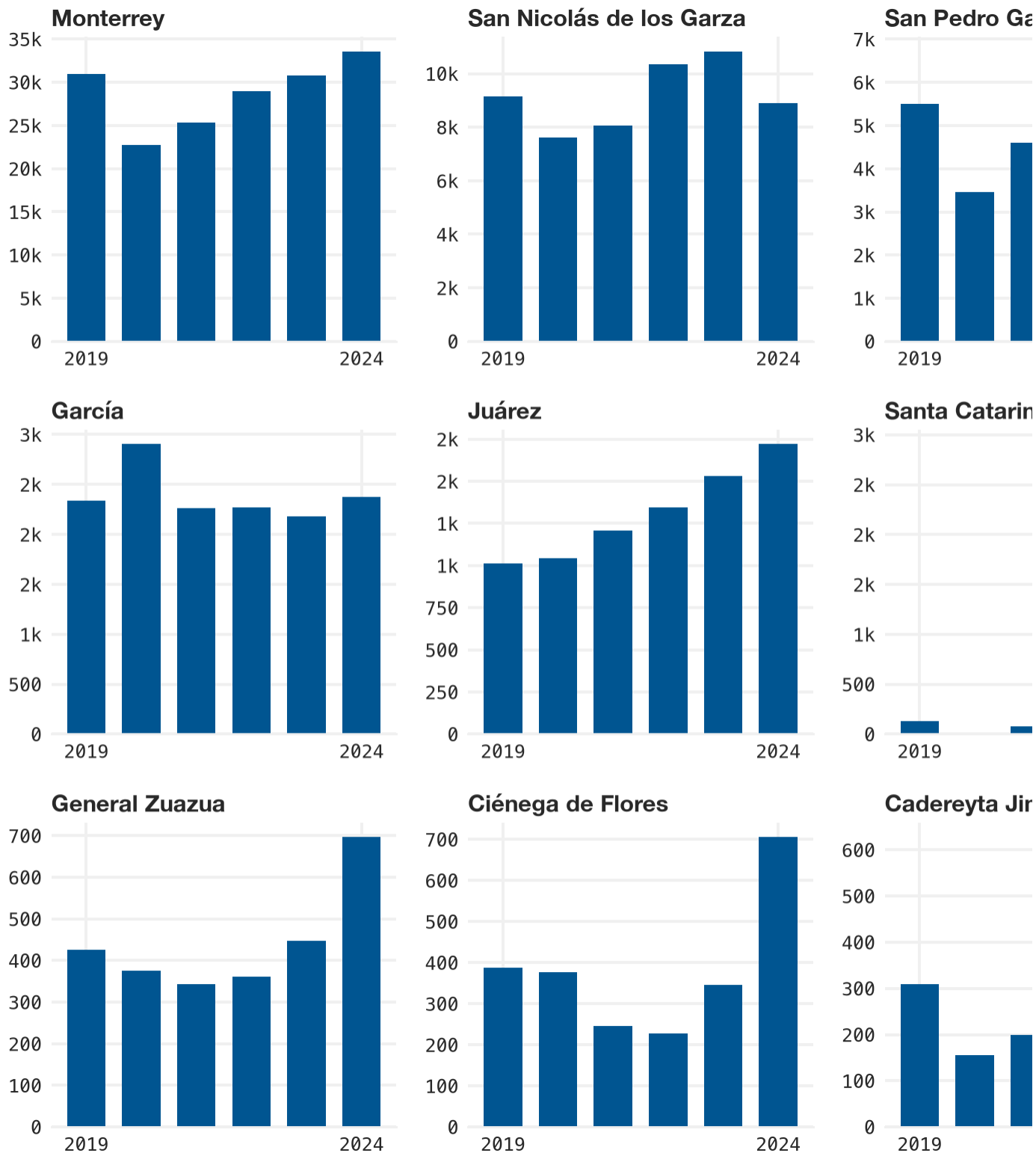
Las series municipales

Para entender de dónde sale esto hay que mirar las series por municipio, que ningún modelo de este notebook usa directamente.

```
serie_mun = union.pivot_table(index="NOM_MUN", columns="ANIO", values="hex", aggfunc="siz
serie_mun["2024 / 2023"] = serie_mun[2024] / serie_mun[2023]
serie_mun["2024 / media 19-23"] = serie_mun[2024] / serie_mun[ANIOS[:-1]].mean(axis=1)
serie_mun.sort_values("2024 / 2023").style.format({a: "{:,.0f}" for a in ANIOS} | {"2024
```

ANIO	2019	2020	2021	2022	2023	2024	2024 / 2023	2024 / media 19-23
NOM_MUN								
Abasolo	3	4	6	11	12	3	0.25	0.42
General Escobedo	1,548	1,860	2,270	4,031	4,375	3,565	0.81	1.27
Guadalupe	5,527	8,032	7,028	4,954	3,335	2,721	0.82	0.47
San Nicolás de los Garza	9,145	7,627	8,053	10,354	10,830	8,911	0.82	0.97
Hidalgo	39	84	61	81	102	91	0.89	1.24
Apodaca	4,489	4,242	2,797	2,615	4,055	3,632	0.90	1.00
Santa Catarina	128	3	77	1,795	2,905	2,730	0.94	2.78
San Pedro Garza García	5,507	3,466	4,597	5,900	6,428	6,717	1.04	1.30
García	2,341	2,903	2,259	2,272	2,181	2,374	1.09	0.99
Monterrey	30,944	22,700	25,318	28,954	30,790	33,568	1.09	1.21
Juárez	1,011	1,042	1,209	1,346	1,531	1,720	1.12	1.40
Cadereyta Jiménez	310	156	200	309	530	628	1.18	2.09
Pesquería	499	273	295	547	842	1,093	1.30	2.23
Santiago	432	430	403	572	760	1,160	1.53	2.23
General Zuazua	425	375	344	361	447	697	1.56	1.79
Ciénega de Flores	388	376	246	228	345	706	2.05	2.23
Salinas Victoria	278	287	259	829	351	823	2.34	2.05
El Carmen	87	166	209	76	53	156	2.94	1.32

```
orden = serie_mun[ANIOS].sum(axis=1).sort_values(ascending=False).index
fig, ejes = plt.subplots(3, 6, figsize=(13, 6.2))
for ax, m in zip(ejes.flat, orden):
    ax.bar(ANIOS, serie_mun.loc[m, ANIOS], color=AZUL, width=0.7)
    ax.set_title(m, fontsize=8.5, color=GRAFITO, pad=4)
    ax.set_xticks([2019, 2024]); ax.tick_params(labelsize=7)
    ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda v, _: f"{v / 1000:.0f}k" if v >
    pulir(ax)
plt.tight_layout(); plt.show()
```



Accidentes por año en cada municipio de la ZMM. Cada panel tiene su propia escala: lo que se compara es la forma, no el nivel.

Las series no son de siniestralidad. **Santa Catarina registró 3 accidentes en 2020** y 2,905 en 2023. **Guadalupe registró más en plena pandemia (8,032) que en cualquier otro año**, cuando todos los demás cayeron, y en 2024 lleva un tercio de eso. Salinas Victoria alterna 286, 827, 351, 821. Ninguna dinámica vial produce eso; es la práctica de captura de cada municipio cambiando de un año a otro —el

mismo fenómeno que `tasa_captura` mide para `CINTURON` y `ALIENTO` en la limpieza, ahora en el conteo mismo.

El supuesto transversal "el subregistro es estable en el tiempo por celda" **no se cumple**, y es la fuente dominante del error de predicción. Nada de lo espacial —Moran, LISA, BYM2— compite con esto en magnitud.

---

7. La ventana del histórico

Si los regímenes de reporte cambian, la historia lejana puede ser peor predictor que la reciente. Se ajusta la NB2 con `h_rec` (2022-2023) en lugar de `h`, y con ambas.

```
with warnings.catch_warnings(record=True) as avisos:
    warnings.simplefilter("always")
    m_rec = smf.negativebinomial("y ~ C(mes) + C(periodo) + logh_rec", data=tr).fit(disp=
hessiana_rec = not any(issubclass(x.category, HessianInversionWarning) for x in avisos)
m_ambas = smf.negativebinomial("y ~ C(mes) + C(periodo) + logh + logh_rec", data=tr).fit(
theta_rec, theta_ambas = 1 / m_rec.params["alpha"], 1 / m_ambas.params["alpha"]

resultados.append(evaluar("M4  NB2, historia 2022-23", m_rec.predict(te).values, theta_re
resultados.append(evaluar("M5  NB2, ambas historias", m_ambas.predict(te).values, theta_a
print(f"M4: coef. log h_rec = {m_rec.params['logh_rec']:.3f} "
      f"(convergió: {m_rec.mle_retvals['converged']}; Hessiana invertible: {hessiana_rec}
print(f"M5: coef. log h = {m_ambas.params['logh']:.3f}, coef. log h_rec = {m_ambas.params

tabla = pd.DataFrame(resultados).set_index("modelo")
tabla.style.format({"devianza": "{:,.0f}", "MAE": "{:.3f}", "CRPS": "{:.3f}",
                    "cobertura 90 %": "{:.1%}", "PAI 5 %": "{:.2f}", "PAI 10 %": "{:.2f}"
```

```
M4: coef. log h_rec = 0.997 (convergió: True; Hessiana invertible: False)
M5: coef. log h = 1.038, coef. log h_rec = -0.027
```

modelo	devianza	MAE	CRPS	cobertura 90 %	PAI 5 %	PAI 10 %
M0 histórico 2019-23	40,949	1.232	0.899	89.8%	8.44	5.85
M0r histórico 2022-23	35,843	1.184	0.835	92.0%	8.42	5.96
M1 Poisson	39,917	1.233	0.885	90.3%	8.44	5.85
M2 NB2	39,914	1.240	0.876	93.9%	8.44	5.85
M3 NB2 + rezago espacial	39,913	1.240	0.876	93.9%	8.44	5.85
M4 NB2, historia 2022-23	35,899	1.186	0.838	92.1%	8.42	5.96
M5 NB2, ambas historias	40,135	1.244	0.879	93.8%	8.44	5.84

```
r_rec = rqr(te.y.values, dist_de(m_rec.predict(te).values, theta_rec))
r_rec_hex = pd.Series(r_rec, index=te.index).groupby(te.hex).mean().reindex(H.hex)
mor_rec = Moran(r_rec_hex.values, W, permutations=999)
print(f"I de Moran del residual con historia reciente: {mor_rec.I:.3f} (z = {mor_rec.z_si
```

```
I de Moran del residual con historia reciente: 0.197 (z = 12.8), contra 0.303 con
historia completa
```

Dos cosas, las dos importantes:

- **La historia reciente gana en todo lo que mide la distribución**: la devianza baja un 10 %, el CRPS de 0.876 a 0.838, y el I de Moran del residual de 0.30 a 0.20. Dos años de datos predicen 2024 mejor que cinco, porque los dos últimos comparten régimen de reporte con el año de prueba.
- **Cuando se le dan las dos, el modelo elige la completa** ($\beta_h = 1.04$, $\beta_{h_{rec}} = -0.03$) y pierde. Eso es un artefacto: en entrenamiento la historia completa explica mejor 2019-2021 *porque los incluye*, y la verosimilitud premia eso aunque no sirva para 2024. Es el ejemplo de por qué el AIC no puede comparar ventanas de historia y por qué solo cuenta la validación fuera de muestra.

M4 converge, pero su Hessiana no se invierte: el ajuste no da errores estándar. La comparación usa solo sus predicciones y el coeficiente puntual, que no los requieren.

El *baseline* crudo con historia reciente (M0r) empata o supera a la NB2 con la misma historia (M4). La estacionalidad y el periodo aportan poco sobre un promedio bien elegido; el régimen municipal aporta mucho, y ninguno de estos modelos lo tiene.

---

8. Qué tiene que ser el paso 5

Este notebook se hizo para decidir la especificación del modelo jerárquico. Lo que decide:

Verosimilitud: Binomial Negativa. La sobredispersión condicional es real ($t = 60$), NB2 sobre NB1, y no hace falta inflación de ceros: los ceros en exceso son municipios sin reportar, no un segundo proceso.

Jerarquía: celda ⊂ municipio, con el municipio variando en el tiempo. La estructura del error no es espacial fina, es municipal. El modelo necesita un efecto por municipio y periodo —una caminata aleatoria por municipio, o al menos un efecto municipio × año— que capture el régimen de reporte. Un BYM2 sobre celdas sigue teniendo sentido para el *shrinkage* de las celdas con pocos datos (la mitad tiene menos de 30 accidentes en seis años), pero es el término secundario, no el principal.

Historia: la reciente, o mejor, ninguna como covariable. En un jerárquico la "historia" es el efecto aleatorio de la celda, estimado de todos los años con los pesos que la verosimilitud le dé; no hace falta pre-cocinarla. Lo que sí hay que decidir es cuánto pasado usar, y este notebook dice que el pasado lejano estorba.

Validación: la misma tabla. M0r es la cota nueva: CRPS 0.835, PAI 8.4. El paso 5 tiene que superar eso en CRPS *y* mover el PAI, que hasta ahora ningún modelo ha movido.

Un límite que ningún modelo cruza. Que Santa Catarina empezara a reportar en 2022 o que Guadalupe dejara de hacerlo en 2023 no está en los datos anteriores a que pasara. Un modelo con efecto municipal *persistente* predice bien mientras el régimen no cambie, y se equivoca el año que cambia. Eso no es un defecto del modelo: es el techo que impone medir siniestralidad con reportes administrativos, y hay que decirlo en cada entrega.

Conclusiones

Sobre la dispersión. La razón varianza/media pasa de 12.9 marginal a 1.37 condicionando en la historia de la celda; lo que queda es significativo ($t = 60$) y la Binomial Negativa NB2 ($\theta = 5.5$) lo corrige. NB2 supera a NB1 y a Poisson por AIC y por CRPS.

Sobre los ceros. Hay un 2.6 % de ceros en exceso. No están dispersos: ocho pares municipio × periodo explican dos tercios, con Santa Catarina 2019-2021 a la cabeza. Son celdas cuyo municipio no reportaba, no celdas sin riesgo. Se descarta ZINB.

Sobre el ranking. Ningún GLM con covariables temporales cambia *dónde*: con $\beta_h = 1$ el orden de celdas es el del histórico (Spearman 1.0) y el PAI se queda en 8.4. Los GLM mejoran cuánto y con qué certeza, no dónde.

Sobre el residual. Tiene autocorrelación espacial (I = 0.30), pero es casi enteramente municipal: 0.06 al quitar la media por municipio. El error de 2024 va de 0.43 (Guadalupe) a 2.5 (Santa Catarina) en cociente observado/predicho, y las series anuales por municipio muestran por qué: cambian de régimen de reporte de un año a otro.

Sobre la historia. Dos años recientes predicen mejor que cinco (CRPS 0.838 contra 0.876). El promedio crudo de 2022-2023 es la cota a superar: CRPS 0.835.

Hallazgo nuevo de este notebook: el supuesto de subregistro estable por municipio es falso y es la fuente dominante del error predictivo. Santa Catarina registró 3 accidentes en 2020; Guadalupe registró su máximo en plena pandemia y cayó a un tercio en 2024. Cualquier modelo de esta base es un modelo de *reportes*, y la jerarquía tiene que llevar al municipio como nivel con efecto temporal propio.

Advertencia metodológica

La covariable `log h` se construye a partir de la respuesta en entrenamiento. Eso la hace un buen predictor y un mal término para comparar por AIC: dos ventanas de historia distintas dan verosimilitudes que no miden lo mismo. En este documento el AIC solo se usa entre modelos con la misma `h`; todo lo demás se decide en 2024.